

Training a Code LLM from Scratch · Part 5: Pivoting to the Gemma 4 Developer Agent — Standing Up the Eval Environment

English · V1 · 2026-10-05

0. Why Part 5

Parts 1–4 pretrained a 0.6B Transformer from scratch, ran SFT, and derived the theory of distributed training and RL. The conclusion: the 0.6B ceiling is set by model scale, not by technique. To get a model that actually *solves* tasks, the right move is **a ready-made strong base + post-training (LoRA / RL)**.

Kaggle’s *Google – The Gemma 4 Developer Agent Competition* fits exactly: the base is fixed and the task is SWE-bench-style bug fixing — a natural hands-on arena for the RL theory from Part 4. Part 5 covers everything from “understanding the competition” to “the local eval harness running its first task”: concepts, architecture, and every pitfall along the way.

Entered both tracks: main (final 12-02) and Paper track (11-12). The goal is learning RL + agentic systems and producing a paper / blog, not chasing rank.

1. What the competition actually is

- **Fixed base:** `gemma-4-31b-it-qat-w4a16-ct` — 31B, 4-bit weights / 16-bit activations (QAT). The base is not part of the submission and cannot be swapped.
- **Task:** an agent in an offline sandbox navigates a Python repo → localizes the bug → edits code → submits a git diff. Scoring is SWE-bench-style PASS/FAIL; the metric is Resolution Rate.
- **Public tasks:** 129 tasks from only 4 repos (fastapi / rich / requests / httpx), each at a different commit. Kaggle scores on a separate hidden set (private repos).
- **Submission:** `submission.zip` < 3 GiB = `agent.yaml` (entry point, purely declarative) + prompts + sub_agents + skills + optional LoRA. No Python entry points allowed.
- **Eval hardware:** 4×L4 (96 GB total), 32K context, default per-task budget of 60 min / 100 tool calls / 500 turns.
- **Two-phase eval:** in Container A the agent edits code and calls `submit_patch`; in Container B the patch is applied, **test files are reset to baseline** (anti-cheating), the official `test_patch` is applied, and pytest runs.

- **9 tools:**

`run_command` / `read_file` / `edit_file` / `write_file` / `get_status` / `submit_patch` / `get_code_neighbors` / `search_similar_code` / `get_code_subgraph`.

- **Gotcha:** `search_similar_code` is offline; the query must be a **symbol name** (class / function), not natural language — it matches the query against node keys in the `.npz` and ranks by cosine similarity.

2. RL derivation: why GRPO, and what to do about multi-step

2.1 V, Q, A: the critic only estimates V

- $V(s)$: expected total reward from state s to the end under the current policy.
- $Q(s, a)$: expected total reward if we take action a in s , then follow the policy.
- $A(s, a) = Q(s, a) - V(s)$: how much better this action is than “average”.

In actor-critic, the policy network outputs action probabilities and samples; the critic estimates $V(s)$. **Q needs no separate network** — it is assembled from the realized reward plus the next state’s V :

$$Q(s, a) \approx r + \gamma \cdot V(s'), \text{ hence } A \approx r + \gamma \cdot V(s') - V(s) \text{ (the TD error)}$$

Accumulating TD errors with $\gamma\lambda$ weighting gives GAE (the one hand-written for CarRacing).

2.2 Single-step: V collapses into a baseline

A single-step task (e.g. one-shot code generation) has no s' , so $Q(s, a) = r$. $V(s)$ is not meaningless — it collapses into “expected reward over actions”, i.e. the **baseline**:

$$A = r - \text{mean reward}$$

GRPO: sample N responses for the same prompt and **use the group mean reward as the baseline**, with no critic:

$$A_i = (r_i - \text{mean}(r)) / \text{std}(r)$$

Combined with RLVR (verifiable rewards), the reward for code tasks comes straight from tests, so no reward model is trained either.

2.3 This competition forces GRPO

The submission contains only `agent.yaml` + LoRA — i.e. only the policy — and evaluation only loads the policy to produce a patch. **There is nowhere to put, and no way to train, a critic**, so PPO is out; only critic-free GRPO-style methods apply.

2.4 Multi-step: trajectory-level reward, A broadcast to every step

A SWE agent is a multi-step MDP: navigate → read → edit → test → fix, with reward only at the end (does the patch pass). GRPO in the multi-step setting:

1. Sample N full trajectories for the same task;
2. One reward per trajectory, baseline computed within the group;
3. $A = (\text{trajectory reward} - \text{group mean}) / \text{std}$, and **every step and every token of the trajectory gets the same A**.

The cost is coarse credit assignment: you can't tell which of dozens of steps mattered. PPO + critic can do per-step credit via GAE; here we can't.

Key distinction: **multi-step without a critic is a constraint of this competition (its submission / eval format), not a limitation of RL methods**. In a self-built multi-step RL setup, PPO + critic is still on the table.

2.5 Multi-step is not combinatorial explosion

- **Depth**: trajectory length is bounded by budget (100 tool calls / 500 turns / 60 min) — dozens of steps, not two or three levels. It's sequence length, not tree depth.
- **Not tree search**: GRPO samples N **independent linear trajectories**, each run start to finish without branching at every step, so there is no combinatorial blow-up.
- **No replay buffer**: GRPO / PPO are on-policy — sample a batch → update → discard → resample. Experience replay belongs to off-policy methods like DQN.

2.6 Both scaffold and RL

Scaffold and RL are not either/or: the scaffold (yaml + prompts + tool orchestration) provides the wiring; RL / LoRA trains the decision weights. Order:

1. Minimal raw scaffold → baseline score;
2. Prompt tuning (like hyperparameter tuning — no weight changes, fast iteration);
3. Once the scaffold is decent and yields positive samples, add RL.

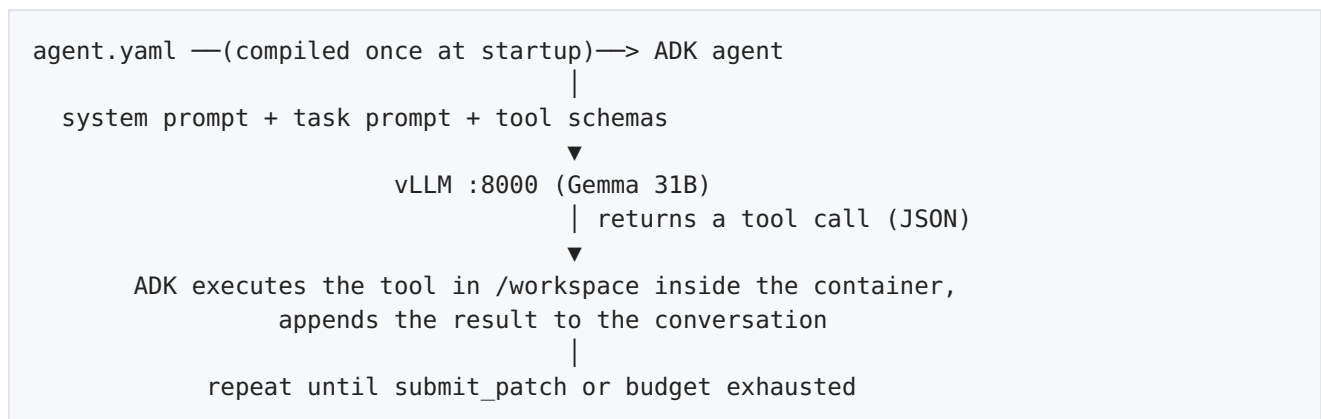
3. System overview: how one task runs end to end

3.1 Three libraries + one inference server

Component	Role
swegemma	The harness: two-container lifecycle, binds the 9 tools, builds the task prompt, extracts the diff, scores with pytest. CLI: <code>swegemma eval</code>
adk-submission	Compiles <code>agent.yaml</code> into a Google ADK agent (never executes contestant code); can manage the vLLM process
adk-eval-core	Lower layer: sandboxes, 3-tier fault-tolerant <code>edit_file</code> , token accounting, trajectory traces
vLLM	Wraps Gemma as an OpenAI-compatible HTTP service (<code>127.0.0.1:8000/v1</code>)

In one line: **harness = exam hall + proctor + grader**. The agent is the examinee, the model is its brain. We control only the examinee (prompts, tool selection, sampling params, LoRA); the harness is fixed and identical locally and on Kaggle.

3.2 One loop



Easy-to-confuse points:

- **The yaml is used once at startup** and defines what the agent *can* do. At each step the model emits a tool call (JSON) whose name must appear in the yaml's `tools:`.
- **Two prompts:** the system prompt is ours; the first user message (task prompt) is produced by the harness's `build_agent_prompt` from a fixed template + repo name + issue text + environment notes. We cannot change it.
- **/workspace lives inside the container:** each task gets a fresh container with the snapshot extracted to `/workspace`, destroyed afterwards.
- If the agent never calls `submit_patch`, the harness extracts the diff as a fallback; `submit_patch` and `get_status` don't count toward tool calls.
- Scratch scripts belong in `/tmp` — anything left in `/workspace` ends up in the patch.

3.3 Data: snapshot / graph / embedding

- **snapshot** (.tgz, one per task, ~220 MB): the full git repo at base_commit, truncated via `git fast-export` — **no future commits**.
- **graph** (NetworkX JSON, per commit): a code relationship graph precomputed by the organizers via static analysis. Nodes are modules / classes / functions; edges are `CALLS` / `DEFINED_IN` / `IMPORTS`. Backs `get_code_neighbors` / `get_code_subgraph`.
- **embedding** (256-dim .npz, per commit): a vector per graph node, backing `search_similar_code`.
- 129 snapshots but 127 graphs / embeddings: a few tasks share a commit.
- Each `tasks.jsonl` row: `instance_id` / `repo` / `base_commit` / `patch` (gold) / `test_patch` / `problem_statement` / `hints_text` / `created_at`. Gold patch and `test_patch` are for analysis only — never put them in a prompt.

4. Environment setup log

4.1 Choosing the machine

- **RunPod doesn't work**: a pod is itself a container with kernel-level namespace / capability restrictions, so docker can't run.
- **Lambda Labs**: a full VM, A100-SXM4-40GB, \$1.99/hr. Docker + GPU validated, so the full harness runs locally without waiting on Kaggle.
- **Persistence**: a Persistent filesystem mounted at `/lambda/nfs/gemma-comp` (\$0.20/GB/month) survives instance termination. Local disk and `/var/lib/docker` do not.

Principle: data, wheelhouse, venv, and model weights all live on NFS; a new machine only needs `bootstrap.sh`.

4.2 Steps and pitfalls

Step	Pitfall / conclusion
List competition files	Slug is <code>gemma-4-developer-agent</code> ; a guessed name returns 404. The file list is paginated (524 files, 6 pages)
Download small files	<code>README</code> , <code>tasks.jsonl</code> , <code>docker/</code> , <code>sandbox/</code> , <code>sample_submission/</code> — 17 files
Read <code>sample_submission</code>	A full demo: main agent + <code>code_analyzer</code> sub-agent; the two LoRAs are ~200 KB placeholder adapters; <code>eval_config</code> is a 1-min / 10-tool-call smoke config; <code>system.md</code> is a ~4 KB detailed workflow — none of it is raw
Install the harness	The three libraries are not on PyPI ; they ship in the Kaggle dataset <code>metric/gemma-4-developer-agent-wheelhouse</code> . Use <code>--find-links</code> , not <code>--no-index</code> (the wheelhouse lacks public deps)
Python version	System is 3.10; uv creates a 3.12 venv

Step	Pitfall / conclusion
venv on NFS	uv cache and venv are on different filesystems, so hardlinks fail and it falls back to full copies; installing swegemma took 6 min. Kept on NFS anyway: slow once, saves a lot on every new machine
vLLM	Not a swegemma dependency; the wheelhouse ships 0.19.1 separately. Installing it downgraded torch 2.14.1 → 2.10.0+cu128; driver 580, CUDA OK
Model weights	HF google/gemma-4-31B-it-qat-w4a16-ct, 23.3 GB, not gated. HF_HOME on NFS; ~1 min to download
vLLM startup error	swegemma pulled transformers to 5.18.0, which made head_dim a per-layer attribute; vllm 0.19.1 reads config.head_dim and crashes. Fixed by picking the latest release before vllm's date (04-18): 5.5.4 (04-13)
Who starts vLLM locally	Confirmed in source: local swegemma eval does not start vLLM; it connects to localhost:8000/v1 and requests openai/gemma-4-31b-it-qat-w4a16-ct. So we start it ourselves, and --served-model-name must match
Launch args	Generated by calling adk_submission.server.VllmServer.build_cmd() directly, matching Kaggle (tool / reasoning parser = gemma4, thinking on, 32K); only TP changed to 1, plus served name
Docker permissions	User wasn't in the docker group; the harness itself must reach docker, so sudo isn't a workaround
Build the image	Dockerfile.sandbox build context is the docker/ directory. Default image name swebench-sandbox:latest
Offline wheels	Installing the repo-under-test's deps in the container needs the competition's wheels/ (124 files), downloaded separately

4.3 VRAM budget (A100-40G, single GPU)

Kaggle uses 4×L4 at 24 GB each. 23 GB of weights on one L4 leaves almost nothing for KV cache, hence tensor parallel = 4: each layer of the *same* model is split into 4 shards and all 4 GPUs compute every token together. An A100-40G holds it on one card, so TP=1 with identical behavior.

Measured from the vLLM log:

Item	Value
Weight memory	19.62 GiB
Available KV cache	14.71 GiB (~16,064 tokens)
Max concurrency at 32K/request	3.03×
Weight loading	175 s (~130 MB/s from NFS)
Profile + warmup	131 s
Attention backend	Heterogeneous head_dim (256 / 512) → forced TRITON_ATTN

Conclusion: 40G is **enough for inference** (baseline, prompt tuning, local concurrency 1-2) but **not for RL training** (base + activations / gradients of long trajectories, competing with rollouts for the same GPU). For RL, switch to 80G or multi-GPU — with everything on NFS, it's just a bootstrap run.

5. Concept cheat sheet (plain language)

- **vLLM**: treats the LLM as a service. Loads weights onto the GPU and exposes an OpenAI-format HTTP API; the agent sends one request per step. Strengths: PagedAttention, continuous batching, multi-LoRA, prefix caching.
- **Uvicorn / FastAPI**: FastAPI defines “how each URL path responds” and validates requests via type annotations; Uvicorn is the server that actually listens on the port. Seeing `Uvicorn running / Application startup complete` means the service is up. (The fastapi tasks in the dataset fix bugs in this very library.)
- **CUDA graph**: pre-records the hundreds of GPU kernel launches of one forward pass as a “macro”; afterwards the CPU just says “play”. The GPU does the math, but the CPU dictates every step; at small batch sizes, CPU launch overhead is the bottleneck. The minute or two at startup is spent recording these macros.
- **Docker image / container**: an image is like a “system disk snapshot” with everything installed; a container is a running instance of it. Like a Chrome tab: isolated from each other, sharing one kernel, so it starts in seconds — much lighter than a VM.
- **Harness**: run framework + test framework.

6. Raw baseline

```
name: raw_agent
model: gemma-4-31b-it-qat-w4a16-ct
instruction: !include prompts/system.md
tools: [run_command, read_file, edit_file, write_file, get_status,
        submit_patch, get_code_neighbors, search_similar_code,
        get_code_subgraph]
generate_content_config: !include configs/sampling.yaml
```

- The system prompt is one sentence: you are a software engineer; resolve the issue by editing source under `/workspace`; call `submit_patch` when done. Identity, goal, and terminal action only — no workflow — to measure the model's bare ability.
- Sampling follows the official config: temperature 0.2, top_p 0.95, max_output_tokens 16384, thinking_budget 4096.
- No adapter, sub-agent, or eval_config; budgets set via CLI to match Kaggle: 100 tool calls, 60 min.
- Smoke test: a direct vLLM request parses the tool call correctly into `run_command({"command": "ls /tmp"})`.

- First task: `fastapi_15661` (first line of `tasks.jsonl`, chosen only to get the pipeline working), now running; system / task prompts were injected successfully.

Result pending: the first task was still running at the time of writing.

7. Next steps

1. Get the `fastapi_15661` result, then the full 129-task raw baseline Resolution Rate.
2. Write `debug_case.py`: given an `instance_id`, extract the repo and place the gold / test / agent patches side by side; render the trace as readable markdown (“thinking → tool call → result”); start an interactive container from the same image to reproduce scoring.
3. Prompt tuning, iterating from case analysis.
4. Once the scaffold yields positive samples, GRPO + LoRA on an 80G GPU.

Appendix: NFS layout and bootstrap

```
/lambda/nfs/gemma-comp/  
├─ bootstrap.sh           # restore env on a new machine  
├─ secrets/kaggle.json  
├─ venv/                  # py3.12: swegemma 0.2.7, vllm 0.19.1, transformers 5.5.4  
├─ wheelhouse/           # official harness wheels  
├─ hf/                    # HF_HOME, model weights  
├─ data/                  # tasks.jsonl, snapshots/, graphs/, embeddings/, wheels/,  
docker/ ...  
├─ submissions/raw_v0/  
├─ results/raw_v0/       # summary.json, patches/, traces/, logs/  
├─ repos/                 # extracted repos for human inspection  
└─ logs/vllm.log
```

What `bootstrap.sh` does: install `uv` and the same Python 3.12.15 (the `venv`’s python is a symlink into `~/.local`, which breaks on a new machine) → restore kaggle credentials → write env vars (`HF_HOME`, `PATH`, ...) → docker permissions → build the image if missing → ensure transformers is 5.5.4 → self-check. It assumes the new machine is in the same region with the same filesystem attached; cross-region means starting over.